

SDP



GitHub Copilot SDK

Build AI-Powered Tools for Your Workflows

Stop building AI infrastructure. Start building the thing that matters.

"What if shipping an AI automation took 10 lines of code — not a month of infrastructure work?"

The Copilot SDK puts the full production agent runtime in your hands.
Same engine. Your workflows. Instant value.

Platform Engineers

Wire Copilot into CI, cron jobs, and internal tools without building an AI stack

2 hours → ~15 minutes

Release notes automation — first pattern most teams ship

Developers

Ship AI-powered automations this sprint — release notes, code review, test analysis

45 min per failed CI run

Manual test-failure analysis eliminated by the scheduler pattern

Architects

Evaluate SDK trust posture: BYOK, tool permissions, model routing, MCP integration

30 min/PR → zero

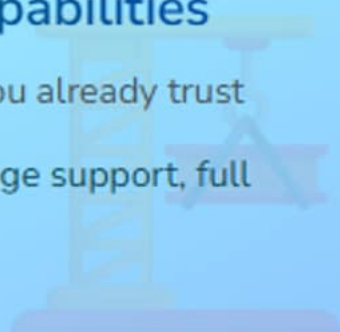
Code standards enforcement automated end-to-end

PART 1

Architecture & Capabilities

The production runtime you already trust

JSON-RPC façade, language support, full capability map



PART 2

Getting Started

10 lines to a working agent

Install, CopilotClient, run_agent — first automation live

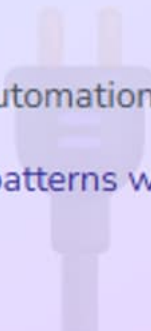


PART 3

Integration Patterns

CLI → Web API → Scheduled Automation

Production-grade deployment patterns with enterprise controls

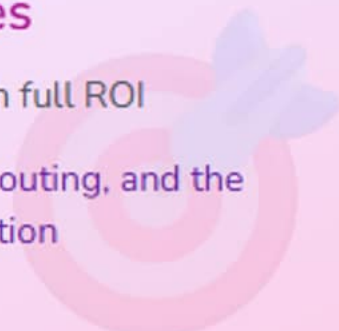


PART 4

Advanced Features

Real-world use cases with full ROI

BYOK, streaming, model routing, and the numbers that justify adoption



Click any section to jump directly there

Architecture & Capabilities

Not a wrapper — the same production agent runtime that powers Copilot CLI, exposed as an SDK.



JSON-RPC over stdio

Thin protocol façade over the battle-tested CLI agent



Four Languages

Python, TypeScript/Node.js, Go, .NET — same API surface



Full Runtime Included

Planning loops, tool orchestration, MCP, BYOK — all ship with the import

```
import CopilotClient
```

↳ Months of production runtime ship with that one line

SDK = Production CLI Runtime + Protocol



JSON-RPC over stdio

SDK spawns Copilot CLI in server mode and manages the protocol — no custom AI runtime needed



Planning Loops

Multi-turn agentic execution with full tool orchestration — identical to what Copilot Chat uses



Tool Orchestration

File system, terminal, web, MCP servers — all available inside your SDK agent calls



Context Management

Conversation state, working directory scoping, and permission controls built in from day one

Four Languages, One Production Runtime



Python & Node.js

Tier-1 support — most SDK examples, fastest iteration, largest ecosystem overlap with AI tooling

```
pip install copilot-sdk
```

```
npm install @github/copilot-sdk
```



Go & .NET

First-class support for enterprise platform teams already in the Go or C# stack

```
go get github.com/github/copilot-sdk
```

```
dotnet add package  
GitHub.CopilotSdk
```



Same Runtime Everywhere

All four languages wrap the identical CLI agent binary — no behavioral differences across stacks

Everything That Ships With the Import



MCP Integration

Connect any MCP server — tools surface automatically inside the agent's execution loop



BYOK Support

Bring your own API key for cost governance and enterprise compliance out of the box



Model Routing

Route requests to the right model per task — balance cost, latency, and capability



Streaming

Real-time token streaming for long-running agents — no timeout cliff, responsive UX



These are not add-ons — they are part of the CLI runtime you import.

Getting Started

Install once, call `run_agent` — the full agentic execution loop in under 10 lines of code.



One Install Command

`pip install` or `npm install` — no infra setup



CopilotClient + `run_agent`

10 lines from install to working agent call



Release Notes in 15 min

First automation most teams ship — same day

```
pip install copilot-sdk && python hello_agent.py  
↳ 2-hour release notes job → ~15 minutes automated
```

The Full Agentic Loop in 10 Lines

```
from copilot_sdk import CopilotClient

client = CopilotClient()

result = client.run_agent(
    prompt="Summarize changes in CHANGELOG.md",
    working_dir="/repo",
    allowed_tools=["read_file", "list_files"]
)

print(result.output)
```

🔄 Full planning loop

Multi-turn tool use — not a single LLM call

🔒 Tool allowlist

`allowed_tools` scopes what the agent can touch

📁 Working dir scope

Agent's file access stays inside `/repo`

Release Notes: The Team's First Automation

One SDK call replaced two hours of manual engineering work

>85%

time reduction

2 hours → ~15 minutes per release

📁 Before

Engineer reads commits, drafts notes, reviews, publishes — 2 hours per release

⚡ After

SDK agent reads changelog, formats release doc — 15 minutes automated

🚀 Ship It Today

Runs from the 10-line pattern you just saw — no extra infrastructure

💡 The SDK agent does the same planning loop a human would — read, reason, write — just consistently and instantly.

Integration Patterns

From one-shot CLI tool to persistent HTTP service to autonomous cron job —
three production-grade patterns.



CLI Tool

Wrap run_agent in a script — simplest entry point



Web API

Flask/Express endpoint — agent on demand, HTTP-native



Scheduled Automation

Cron + SDK — fully autonomous, zero human intervention

```
cron: 0 * * * * python ci_analyzer.py  
↳ 45-min manual CI failure analysis → automated on every run
```

From Script to Persistent Service

CLI Tool Pattern

One command, one agent call

```
python analyze_pr.py --pr 1234
```

Pipe output to CI

Exit code drives pass/fail gates

Perfect for GitHub Actions steps

Web API Pattern

Flask or Express endpoint

POST /analyze → streaming response

Webhook-driven

PR opened → agent fires automatically

Multi-user, persistent, HTTP-native

```
@app.route("/analyze", methods=["POST"])
def analyze():
    return client.run_agent(prompt= ... )
```

Fully Autonomous AI on a Cron Schedule

❌ The Problem

45 min per failed CI run

Engineer reads logs, cross-references tests, files issue

Happens at 2am — no one available

Pattern repeats every broken build

💡 The Solution

Cron triggers SDK agent on CI failure

Agent reads logs, identifies pattern, files issue

```
client.run_agent(  
  prompt="Analyze CI failure  
  allowed_tools=["read_file",  
  )
```

✅ The Outcome

Failure triaged before morning standup

Zero human intervention required

45 min
→ automated

Production Safety: Permission-First Design



allowed_tools

Explicit allowlist of tools the agent can invoke — least privilege enforced at the SDK level

```
["read_file", "list_files"]
```

No shell exec unless declared

Auditable at call site



working_dir Scope

Agent file access restricted to the declared working directory — no escape, no surprise writes



Enterprise Ready

BYOK + permissions + model routing — the security posture InfoSec needs to approve production use

Three Patterns, One Progression

✗ Without SDK

- 1 Build custom LLM wrapper
- 2 Hand-roll tool orchestration
- 3 Manage conversation state
- 4 Handle streaming + retries



Weeks of glue code before first value

✓ With SDK

- 1 `pip install copilot-sdk`
- 2 `CopilotClient() + run_agent()`
- 3 Choose pattern: CLI / Web / Cron
- 4 Ship to production this sprint



Production automation in one sprint

Same four automations. One SDK call each.

Advanced Features

BYOK, streaming, model routing, and MCP — proved through use cases, not listed as feature bullets.



BYOK + Cost Control

Your API key, your model budget,
enterprise compliance



Streaming + Routing

Real-time UX + right model for each
task



ROI Sweep

>85% release notes, CI analysis
automated, zero PR review time

Full ROI metrics across four real automations

↳ 2h→15min · 45min→auto · 30min/PR→zero · 30min logs→auto

Cost Governance and the Right Model Per Task

🔑 BYOK: Bring Your Own Key

Your API key, your invoice

Cost attribution stays inside your org

Enterprise compliance

Data stays on your approved provider

InfoSec approves, finance tracks, team ships

🎯 Model Routing

Fast model for simple tasks

File listing, formatting, categorization

Powerful model for complex tasks

Architecture review, cross-repo analysis

Route per call — optimize cost + quality

Real-Time Output for Long-Running Agents

```
for chunk in client.run_agent_stream(  
    prompt="Analyze all PRs from last sprint",  
    working_dir="/repo",  
    allowed_tools=["read_file", "list_prs"]  
):  
    print(chunk.text, end="", flush=True)
```

⚡ No timeout cliff

Stream tokens as they arrive — works across analysis jobs that take minutes

💻 Responsive UX

Users see progress in real time — no frozen UI waiting for a batch result

🔑 Same permission model

`allowed_tools` and `working_dir` apply identically to streaming calls

Connect Any MCP Server to Your Agent



MCP Servers as Tools

Register any MCP server — tools surface inside the SDK execution loop, zero glue code needed



Example: Database MCP

Agent reads query results, correlates with code, files tickets — all in one `run_agent` call

Read DB schema

Cross-reference commits

Auto-file issue



Example: API MCP

Agent calls your REST API via MCP to pull context that would otherwise require custom tool code

Four Automations, Proven Numbers

Manual Status Quo

Release engineering

2 hours per release — read commits, draft, review, publish

CI test analysis

45 min per failed run — log reading, pattern matching, ticket filing

Code review standards

30 min per PR — enforce style, security, coverage manually

Incident response

30+ min log correlation — pull logs, cross-reference, root-cause

SDK Automated

Release engineering

~15 minutes — agent reads, drafts, formats, publishes

CI test analysis

Automated on every run — pattern detected, issue filed, zero wait

Code review bot

Zero manual time — webhook fires agent on PR open

Incident response

Automated log correlation — root cause in seconds, not 30 minutes

>85%

release time reduction

45 min

CI analysis → automated

0 min

PR standards review time

From AI Infrastructure Work to Shipping Automations

Before

Months building custom LLM orchestration layers

Hand-rolling tool call loops and retry logic

Managing conversation state and context windows

No enterprise controls — approval blocked by InfoSec

After

Production agent runtime imported in one command

Planning loops, tools, and streaming included

BYOK + tool permissions + model routing built in

First automation shipping this sprint

>85%

release engineering time reduction

45 min

CI failure analysis → fully automated

Jan 2026

Technical Preview — install in 15 minutes today

✓ What You Can Do Today

Today

- ❑ Install the SDK: `pip install copilot-sdk` or `npm install @github/copilot-sdk`
- ❑ Run the 10-line hello world against a real repo directory
- ❑ Read one CHANGELOG.md with `run_agent` — see the runtime in action

This Week

- ❑ Pick one repetitive workflow: release notes, CI triage, or PR standards
- ❑ Implement the matching pattern: CLI tool, Web API, or cron script
- ❑ Demo the automation to your team — 15 minutes prep, instant buy-in

This Month

- ❑ Deploy one automation to production with `allowed_tools` and `working_dir` scoped
- ❑ Instrument BYOK for cost tracking and get InfoSec sign-off
- ❑ Identify two more workflows to automate — the ROI compounds with each one

🔑 Key Takeaway

The SDK is in Technical Preview since January 2026 — the runtime is production-tested, the API is stable enough to ship against today.

 Official Resources[GitHub Copilot SDK Repository](#)

Source code, examples, and issue tracker for the SDK

[Build an agent into any app with the Copilot SDK](#)

Official announcement and architectural overview from GitHub Engineering

 Related Tech Talks[Copilot Primitives](#)

Instructions, prompts, agents, and skills — the customization layer the SDK extends

[MCP Apps](#)

MCP server patterns that pair with SDK's native MCP integration

[Copilot Memory](#)

Context and memory patterns relevant to long-running SDK automations



GitHub Copilot SDK

Build AI-Powered Tools for Your Workflows

>85%

release engineering time reduction — 2 hours to ~15 minutes

10 lines

CopilotClient + run_agent — full agentic loop, production runtime

Jan 2026

Technical Preview — install today, first automation this sprint

Which workflow would your team automate first — release notes, CI triage, or PR standards review?